

data to fine tune the BERT based classifier. Another option is to use BERT for feature extraction, wherein you obtain contextual sentence embedding for each sentence from it and use that as input to your hand-built classifier. What will you choose? What is the rationale behind your choice?

10. Transfer learning for different NLP tasks can be done either by using pre-trained word embeddings, pre-trained sentence embeddings or using pre-trained language models like BERT. What are the pros and cons of each of these approaches?
11. What are the differences between traditional recurrent neural network (RNN) architectures and the current transformer based architectures? Why is it that most modern NLP models use transformer based architecture?
12. Can you explain the self-attention mechanism in transformer models? How is this different from encoder-decoder cross attention?
13. What is the difference between hard attention and soft attention? Why do most neural network models use soft attention?
14. What is the major difference between BERT and GPT-2 models? Is it possible to use BERT for text generation? If yes, explain how. If no, explain why it can't be used.
15. Can you explain the difference between word level embeddings, sub-word level embeddings and character level embeddings? What are the pros and cons of each of these three types of input representation?
16. Tokenisation is the fundamental building block in any NLP pipeline. For languages such as English, white space characters/punctuations can be used to guide tokenisation. For other languages, where words are not separated by whitespace/punctuation characters, tokenisation is extremely complex. Can you explain different tokenisation schemes such as byte pair encoding tokenisation and unigram subword tokenisation? Interested readers can refer to a good article on tokenisation available at <https://blog.floydhub.com/tokenization-nlp/>.
17. You are given an iterator of ints of given size 'n'. The only operation you are allowed to use is to return the value of the element pointed to by the iterator. You need to return a random element from the container pointed to by the iterator such that each element has an equal probability of being chosen as the return value.
18. In the previous question, instead of a known size container, you are given an iterator pointing to a container of unknown size. How will you return a random element from the container of unknown size?
19. Most of you must be familiar with regularisation techniques. Dropout can be considered as a form of regularisation in case of deep neural networks. While it is easy to apply dropout in the case of feed forward neural networks, how can you apply it in the case of recurrent neural networks?
20. Given a short text classification NLP model with K classes, can you explain what the confusion matrix for a K class model will look like? Given a confusion matrix

with K classes, how would you identify the opportunities for improving the model?

21. You are given a large dataset of 100,000 samples for a K class text classification problem. But the data set is highly imbalanced, with some classes being represented with a low frequency in it. How will you approach modelling in this case?
22. What are the differences between boosting and bagging? Is it possible to build an ensemble consisting of both types of models? What are the problems and advantages of such an approach?
23. BERT is based on the encoder part of the transformer network. GPT variants are based on the decoder part of the transformer network. What are the advantages of using encoder blocks for pretrained models as opposed to using decoder blocks? What are the disadvantages?
24. Pretrained models like BERT and GPT-3 are huge in terms of the number of parameters they contain. Training such models can be costly in terms of both time and computational resources. There are techniques used to reduce model sizes without sacrificing much of accuracy. Model distillation allows to create smaller models post the creation of a large one, so that the former can be used for inference. But the training overhead still remains. Quantization is another technique which enables representing the model parameters in smaller-sized types (int16 instead of float64), thereby achieving model compression. Can you compare these two techniques — model distillation and model quantization?
25. Model distillation can be used to reduce model size by removing certain parameters and weights of the model. However, this results in the model weight/parameter matrix becoming sparse. GPUs don't handle sparse matrices efficiently. So, how will you improve your approach to work around this issue?

While the above questions went deeper beyond the basics we normally cover in CS interviews, please don't get discouraged if you can't solve all of them. Most interviewers don't go for questions on niche topics, but it would be better to know some of the topical issues in NLP/ML in addition to the fundamental concepts. There is an excellent course on machine learning system design by Chip Huyen at Stanford: <https://stanford-cs329s.github.io/syllabus.html>. I would encourage our readers to go through it.

Feel free to reach out to me over LinkedIn/email if you need any help in your coding interview preparation. If you have any favourite programming questions or software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Wishing all our readers happy coding until next month! Stay healthy and stay safe. 

 **By: Sandya Mannarswamy**

The author is an expert in natural language processing and is currently working as an independent researcher. Her interests include natural language processing, machine learning and AI.